

Question	Answer
what is a frontier model, how is it different from foundation model?	A foundation model is a large pre-trained model that serves as a base for many downstream tasks. Frontier models are simply the most advanced foundation models available at a given time, pushing the state of the art in capabilities, scale, and performance.
Positional Encoding?	Positional Encoding tells the model the order/position of words in a sentence. Without it, "Dog bit man" and "Man bit dog" would look the same to the model. A unique vector is added to each token's embedding based on its position (1st, 2nd, 3rd... word).
Thanks. Is ChatGPT, Claude Sonnet etc are current foundation, frontier models?	Yes, you're correct, they can be considered as frontier models.
Sir, But I heard that BERT is used for Autocomplete, recommending the sentence completion.. is not kind of text generation	BERT is not a generative language model. It is trained using Masked Language Modeling (MLM), where it predicts masked words within a sentence using both left and right context. Autocomplete requires predicting the next token sequentially (left-to-right), which is what decoder-only models like GPT are designed for.
What are the applications of Encoder only models?	Encoder only models (like BERT) are used where you need to understand text, not generate it. Like classification kind of a task
what is meant by WordPiece tokenization?	WordPiece tokenization splits text into subword units instead of whole words. It uses a fixed vocabulary of common words and subwords, allowing it to represent rare or unseen words by breaking them into meaningful pieces. For example: "unbelievable" → ["un", "##believable"] "playing" → ["play", "##ing"] This reduces the vocabulary size while still handling unknown words effectively. The "##" prefix indicates that the subword continues a previous token.
Encoder only models can't predict next token as per what we have discussed. The example taken is to generate next token, can you give some clarity here?	Encoder only models are not used to predict the next token. The Masked token prediction is a task to train such model. BERT can see bi-directional input to predict the masked token. In the actual generational model, model can only see the previous context not the context after that
What does BERT do?	BERT basically learns deep contextual representations of text by considering both the left and right context of every word. It is used for applications like sentiment analysis, sentence similarity etc.
Why are only 15% of tokens masked?	Only 15% of tokens are masked because masking too many tokens makes the sentence difficult to understand. 15% is a good balance that enables the model to learn to predict missing words.
How does BERT predict fox as it can be anything like Tiger, Dog	Based on sentence context, it outputs a probability distribution over its entire vocabulary, and if the probability of the token fox is highest it will predict fox over other tokens
Who decided that what word is supposed to be masked?	This is random while training process.

MLM and NSP seem to be orthogonal training objective. Looks like an encoder only transformer model can't be trained with both objectives, we need to choose one of them. Is this correct?	No actually. Suppose the input is [CLS] The quick brown [MASK] jumps [SEP] It was very agile [SEP] then MLM: Predicts masked word (fox) from the sentence NSP: Uses [CLS] token to predict if sentence 2 follows sentence 1 So total loss = MLM loss + NSP loss
what is CLS embedding?	Suppose you pass a sentence "I love KGP" to BERT model, then BERT will process it as "[CLS] I love KGP", and it will output the embedding of all the tokens, so you will get the token embedding for CLS also
How next sentence prediction is different from Text generation?	Next Sentence Prediction (NSP) is BERT's pre-training task: given two sentences, it's a binary classification predicting whether sentence B actually follows sentence A ("IsNext"/"NotNext"), using bidirectional context. Text generation (decoder-only models like GPT) is autoregressive—predicting the next *token* one at a time using only past context (causal masking), looping outputs back as input to generate entirely new sequences, not just classify sentence relationships.
We learnt about Encoder and Decoder only models previously. I understand BERT is encoder only. Has the decoder only model already have this pretrained encoders as part of their architecture?	No, decoder-only models like GPT or LLaMA do not contain a pretrained encoder inside them. They are completely standalone architectures that use causal self-attention, meaning they are mathematically restricted to only look at past words to predict the next one. From a production standpoint, decoder-only designs dominate modern generative AI because their specific structure allows us to use optimization tricks like KV-caching to keep inference latency low as text is generated.
How do we overcome the drawbacks of the BERT model ?	BERT's key drawbacks—being encoder-only (can't generate text), domain mismatch, and NSP's limited usefulness—are addressed several ways: RoBERTa removed NSP, finding little benefit. Domain-Adaptive Pre-training (DAPT) continues MLM on in-domain unlabelled data before fine-tuning. T5/BART introduced encoder-decoder seq2seq architectures enabling generation. GPT-style decoder-only models solve generation entirely via causal autoregressive prediction, while scaling and instruction-tuning (RLHF) further improved usability.
how can we check similarity/paraphrase/inference on BERT, ideally its encoder model, it wont be able to generate output sentence ... can you please explain	You're partially correct. BERT does not generate sentences. Instead, it encodes the input text into contextual embeddings. These embeddings can then be compared using similarity measures (e.g., cosine similarity) for tasks like semantic similarity or paraphrase detection.
as you said "it will output the embedding of all the tokens, so you will get the token embedding for CLS also" my question is why it generate token for CLS ?	During training, BERT is specifically trained to push all the sentence-level meaning into the [CLS] embedding, So [CLS] embedding becomes a summary of the entire input. And can be used for classification tasks

How do we select the right model for a particular project? Which model is most useful, and how should it be evaluated?	Model selection isn't about finding one perfect model, but rather balancing constraints. Intelligence, Cost, Latency, and Privacy. In production, we highly follow a "tiered routing" architecture where we deploy fast, cheap open-weights models (like Llama or Mistral) for high-volume, simple tasks like intent routing or summarization, and escalate to expensive frontier API models (like Claude Sonnet or GPT) only when deep reasoning is strictly required. To evaluate these models, you can't rely on standard exact-match code tests, you build an "LLM-as-a-judge" pipeline to automatically score responses on specific criteria like Faithfulness (avoiding hallucinations) and Relevance, running these checks directly in your CI/CD workflow.
When we say BERT predicts next sentence, doesn't it mean it would also need a decoder to generate results ?	Please note that it is not next sentence "generation", it is "prediction". It is a binary classification task where the model is given two sentences and predicts whether the second sentence actually follows the first in the original text.
what is zeroshot prompting & multishot prompting?	Zero-shot prompting means giving the model a task with zero examples, while multi-shot (or few-shot) means showing it a few input-output examples before giving it the real task. Think of zero-shot as handing a developer a brand new ticket, and multi-shot as handing them that ticket along with three merged PRs showing exactly how you want the code formatted. In production, zero-shot is your default because it saves token costs and keeps latency low. BUT BUT, you should switch to multi-shot when you need the model to strictly follow a specific format, guarantee a complex JSON schema, or handle a highly domain-specific task where its general knowledge isn't up to the mark.
Can We say that, encoder only models are good for data extraction, from dataset. Like BERT which can be used for classification?	Yes basically encoder-only models are good at understanding & extracting information from text, which can then be used for downstream tasks like classification.
can you help me understand again what is meant by parameters?	You see parameters are simply, the learnable weights of a neural network. During training, the model adjusts these values so it can make better predictions. In general, more parameters increase the model's capacity to learn complex patterns, but they also require more data and compute.
does GPT use a encoder only model to understand the prompt before decoding?	GPT is a decoder only model, it uses only the transformer decoder layers, there is no separate encoder to understand the prompt
Following up on my earlier question: If decoder-only models work so well for everything, why does anything still use the encoder-decoder split like Marian, what advantage does cross-attention provide that we cannot get by just concatenating source and target into one decoder?	While decoder-only models dominate general text generation, the encoder-decoder architecture is still widely used in production for dedicated sequence-to-sequence tasks like translation (e.g., MarianMT) or heavy document summarization (e.g., T5) due to bidirectional context and inference efficiency. In a standard decoder-only model, causal masking prevents the model from looking ahead at the full prompt, whereas an encoder handles the source text with 100% bidirectional visibility. In an encoder-decoder setup, the source text is processed exactly once, creating a static reference sheet that the decoder queries via cross-attention.

In enterprise AI systems, would it be better to combine an encoder model like BERT for retrieval with a decoder model like GPT for response generation, rather than relying on a decoder-only model for everything?	I have been noticing since last 3 sessions, you always ask good questions, very good Abdeali Coming to your question Yes, combining them is exactly how modern enterprise Retrieval-Augmented Generation (RAG) works. This will come in future sessions. If we relied entirely on a decoder model like GPT for everything, we would have to stuff all our company documents into the prompt every time a user asks a question. This is incredibly slow and expensive because compute costs scale heavily with prompt length. Instead, we use an embedding model for retrieval to rapidly search a vector database and extract just the top 3 or 4 relevant paragraphs. We then pass only those specific paragraphs to the decoder model to generate the final answer.
Is decoder only model used for just prediction?	Yes you can use them for prediction. Like you prompt the model "<some movie review>, is the review positive?" then decoder only model produce some thing like "the review is not looking positive something mix or neutral"
Now that decoder-only LLMs dominate everything generative, where does BERT still actually used in production ? is it that for pure understanding tasks like classification or retrieval, ?	Yes. BERT is still widely used for language understanding tasks where text generation is not required. Common production uses include text classification, sentiment analysis, spam detection, named entity recognition, semantic search etc. . Since encoder-only models are typically faster and more efficient than decoder-only LLMs for these tasks, they remain a popular choice.
When we say compute loss in decoder only model- prediction occurs but how the actual word is determined as the goal is to predict only? loss can be calculated when we have both predicted and actual values. please clarify	Good question, and it tells the crucial difference between training a model and using it for inference. During training, we use existing documents where we already know the full text, so the "actual word" is simply the next word written in that specific document. For example, if our training data contains the sentence "The cat sat on the mat", we feed the model "The cat sat on the" and ask it to predict the next word. We then calculate the loss by comparing the model's prediction against the true answer, which we know is "mat". This is called self-supervised learning because the text itself automatically provides millions of perfectly accurate labels without requiring humans to manually annotate them.
while encodes predicts decoder generates, please can you explain the difference?	Lets take an analogy of text summarization, suppose you pass a news article to a encoder decoder model for summarization, encoder first understand the whole news article and generate a intermediate representation, then decoder takes the representation and generate the news summary
Is LAMBADA essentially the decoder-side counterpart to GLUE?	Not exactly. GLUE is a collection of language understanding tasks used to evaluate encoder models like BERT, whereas LAMBADA is a single benchmark that evaluates a model's ability to predict the final word of a passage using long-range context.
Is LAMBADA used only for Decoder model evaluation? Or can it be used in encoder decoder models too	It can also be used to evaluate encoder-decoder models, as long as they can predict the target word or generate text. However, it is most commonly used with decoder-only language models because the task closely matches their next-token prediction objective.

Why did GPT2 perform much better than traditional n-gram language models on LAMBADA?	Traditional n-gram models can only use a short, fixed context, so they struggle with long-range dependencies. GPT-2 uses the Transformer architecture with self-attention, allowing it to consider much longer contexts and capture complex relationships between words. This enables it to make much more accurate next-word predictions on benchmarks like LAMBADA.
thn why cant local context solve LAMBADA?	Local context is often not enough because the correct final word may depend on information mentioned several sentences earlier. LAMBADA is specifically designed so that the last word is difficult or impossible to predict from the immediately preceding words alone, requiring the model to use long-range context across the entire passage.
Follow Up : This is called self-supervised learning because the text itself automatically provides millions of perfectly accurate labels without requiring humans to manually annotate them. Query: So basically here also Model using training data to determine the actual word however since LLM is trained on large data- what if there are multiple actual words that are coming up? does it use further cosine similarity or similar way to come up with actual? Basically what i am having difficulty understanding is - if you see this actual v/s predicted value- actual also is kind of predicted based on training data however it may be wrong also.	Let me clarify this The "actual word" during training is never predicted or calculated by the model. It is completely static and hardcoded it's just the exact next word written in the specific document being processed, even if that document contains typos! If the dataset has "The cat sat on the mat" in one book and "The cat sat on the couch" in another, it processes them separately. It does not use cosine similarity to blend them. Instead, because the model is penalized for missing "mat" in step A and "couch" in step B, the loss function forces the model to learn a probability distribution. Over millions of steps, it learns to assign, say, 45% probability to "mat" and 45% to "couch", which is exactly why LLMs are capable of such diverse and creative generation! Hope its much more clear now Shashank, let me know if you want more examples.
Why two masked self attention in decoder?	Basically there are two: 1. Masked self attention: It is just for the decoder step, Like you generate "I LOVE KGP", for the token "KGP" it only attend "I LOVE" Cross-Attention: It is between encoder (K,V) with decoder (Q), so the while generating KGP it also can attend full context of the encoder
Encoder works on Ngram and decoder uses CBOW ??	CBOW and Skip-gram are training methods used in Word2Vec to learn word embeddings. N-grams are a traditional language modeling approach based on a fixed context window. Transformer encoders (e.g., BERT) and decoders (e.g., GPT) do not use either N-grams or CBOW. They use self-attention to learn relationships between tokens, allowing them to capture much longer-range context than N-gram or CBOW models.
how masking is done random or using some algo	In BERT's Masked Language Modeling, masking isn't done by a complex algorithm, it's largely random, but with a specific procedure. For each training sequence, 15 percent of tokens are randomly selected for masking. Of those selected tokens, 80 percent of the time the token is replaced with the actual MASK token, 10 percent of the time it's replaced with a random word from the vocabulary, and 10 percent of the time it's left unchanged as the original word. This mixed strategy exists because if BERT only ever saw MASK during training, it would learn to expect that token, but MASK never appears during actual fine-tuning or inference. By sometimes replacing with a random word or keeping it unchanged, the model is forced to build a genuine contextual representation for every token, not just rely on spotting the MASK symbol.

Why does BART need both an encoder and a decoder?	BART needs both because of how its denoising pre-training works. The corrupted input (with noise like token masking, deletion, sentence shuffling, or text infilling) is first processed by the bidirectional encoder, which builds a full contextual understanding of the corrupted sequence, similar to BERT. Then the autoregressive decoder takes that encoded representation and generates the original, uncorrupted text token by token, using cross-attention to look back at the encoder's output while reconstructing it, similar to GPT. So the encoder is needed to understand and represent the corrupted input bidirectionally, while the decoder is needed to actually generate clean text as output. Since BART's goal is denoising (input is messy, output is clean text), it needs both halves, the encoder alone (like BERT) couldn't generate output text, and the decoder alone (like GPT) wouldn't get the benefit of bidirectional understanding of the corrupted input before generating.
can u suggest some books i can go through as well? for transformers, llms and for agentic part?	it's better to read the technical blog posts and published papers.
What is a CLS token?	It is a special token added at the beginning of every input in BERT, it is not a real word but just a special marker. You can think [CLS] embedding is the single summary of the whole sentence
The first option might also degrade in quality due to overfit on the small 5000 labelled dataset. Increasing parameters often need an increase in labelled set as well. Is this reasoning correct?	Your reasoning is 100% correct. BERT-Large has 340 million parameters compared to BERT-Base's 110 million. That massive increase in capacity means the model has enough mathematical "room" to simply memorize all 5,000 of your examples instead of actually learning the underlying patterns. In production, if you upgrade to a significantly larger model and do full fine-tuning, you almost always need to scale up your labelled dataset to prevent severe overfitting. Good catch!
could you just summarize T5, GPT2 and bert	BERT: Reads entire sentence at once, best for classification, not for generate free text. Encoder-only T5: Converts every NLP task into text-to-text format, best for translation, summarization. Encoder-Decoder GPT2: Generates text word by word, Best for text generation, autocomplete. Decoder-only
GPT-3 demonstrated that prompting can replace fine-tuning for many tasks. In today's enterprise AI systems so where do we draw the line between using better prompts, RAG, and full model fine-tuning? What factors determine the best choice?	The industry rule of thumb is Use RAG to teach a model new facts, and use fine-tuning to teach it new behaviors. You should always start with basic prompting because it's the fastest and cheapest to iterate. If the model hallucinates because it doesn't know your company's private data, you build a RAG pipeline. This is like giving the model an open-book exam so it can look up the facts at runtime. BUT, if the model struggles with a highly specific format, tone, or complex domain language, or if you want to train a cheap, small model to do one specific task extremely well to save on API costs and latency, that is when you invest the time and compute into fine-tuning.

What is Context-Window?	The context window is the maximum number of tokens (words/subwords) a model can process at once, including both the input prompt and its generated output. It defines how much text the model can "see" and attend to when making predictions. For example, GPT-3 had a context window of 2,048 tokens. Anything beyond that limit gets truncated or ignored. Larger context windows allow models to handle longer documents, maintain longer conversations, and reference more information at once, which is why modern models like Claude 3/4 emphasize long-context capabilities for tasks like analyzing entire books or codebases.
so is the learning hierarchy like this: 1. Zero shot 2. IN context learning OR few shot learning 3. fine-tuning 4. update parameters which essentially changes the model weights and creates an updated model	You're correct, but your point 3 and 4 are functionally same.
Does Fine Tuning means using in context learning methods like zero shot and few shot?	No finetuning change the model parameters but in context learning doesnot change model parameters, only use model for inference
will fine tuning be covered in later sessions?	yes, in the later sessions
We will not be able to reuse in context learning again and again right? Everytime we have to do in context learning because we are not training the model.	Yes, you need to provide the context everytime you use the model
what is PPO?	It is not related to thing discussed now, but briefly PPO used to train your LLM to produced a preffered generation from others. Like ChatGPT used PPO to learn to be helpful, harmless and honest based on human feedback scores
in the Paris example proffessor gives on Paris, does the model predics next word from memory from previous trainings on similar context?	Yes. During inference, the model does not search its training data. Instead, it uses the knowledge stored in its learned parameters (weights) from pre-training.
What is the full form of PPO here	Proximal Policy Optimization
When I give Paris is to chat gpt is gives me output as Paris is Capital of France but Claude is giving response as - Your message seems to be cutting off after "Paris is."	Basically it is design choice, chatgpt may be trained to to be more assumption-forward while claude trained to be more cautious and conversational
So, RLHF the industry standard? But what about the accuracy from human labellers?	Human label quality is indeed a challenge. To improve reliability, companies use multiple annotators, clear guidelines, agreement checks, expert reviewers for specialized domains, and quality audits.
is RLHF a training and dev time only thing, or is there a continous RLHF that can work on a deployed project in production?	RLHF is strictly a training and development phase. You do not update a model's weights live in production. When an LLM is deployed to serve users, its weights are completely frozen. If you tried to run reinforcement learning live on every request, your API latency would skyrocket, compute costs would explode, and the model could easily be manipulated by bad actors. Instead, enterprise teams continuously collect user feedback (like thumbs up/down or text corrections) in production, save it to a database, and use that batch of data to run a new RLHF training job offline at the end of the week or month.

is MMLU accuracy an industry standard, where can we refer realtime data for the same?	MMLU is one of the most well-known academic benchmarks, but it is not the industry standard by itself. Today, models are evaluated on a suite of benchmarks because each measures different capabilities. For example, MMLU tests broad knowledge, GPQA tests graduate-level science reasoning, SWE-bench evaluates coding, and LiveBench focuses on fresh, contamination-resistant tasks. You can refer to platforms like LMArena, LiveBench to get realtime evaluation benchmarks.
If companies use that batch of data to run a new RLHF training job offline at the end of the week or month, isn't the AI serving old data to its users?	RLHF is used to train the tone or style of the generation not the content/information of the generation
So basically serve a model with frozen weights -> collect HF into a DB -> Have a expert review and clean HF -> Batch RLHF on the cleaned and collected HF -> Run RL. Kind of like adding a load balancing on the RL and more monitoring. Is this correct?	Yes your understanding is correct. The main addition is that the cleaned human feedback is first used to train a reward model, and the RL step optimizes the LLM using that reward model rather than the raw human feedback directly.
is reasoning model is different from chain of thoughts model	Yes, they are very different! Chain of Thought is just a prompting technique. It's when we tell a model to think step-by-step, forcing it to print its scratchpad logic into the chat so it has more time to calculate the right answer. A Reasoning Model is a fundamentally different type of model trained via Reinforcement Learning to automatically do that deep thinking in a hidden background process before it ever starts typing to you. Think of CoT as asking an engineer to write their thought process in a Jira ticket, while a Reasoning Model naturally does that deep thinking in its head. From a production standpoint, you should try to avoid CoT prompts on Reasoning Models because it wastes your tokens and can confuse them sometimes.
how many Masked Multi-Head Attention and Feed-Forward layer should be in the decoder model, and what is recommended repetition.	There is no fixed or universally recommended number of Masked Multi-Head Attention and Feed-Forward layers. A decoder is built by stacking identical decoder blocks, each containing one Masked Multi-Head Self-Attention layer and one Feed-Forward Network (plus a Cross-Attention layer in encoder-decoder models). The number of times this block is repeated depends on the desired model size and available compute. For example, GPT-2 Small uses 12 decoder blocks, while GPT-3 (175B) uses 96. Larger models generally use more blocks, but they also require significantly more training and inference resources.
From the Industrial Application Purpose, are the older models like BERT or even GPT-2.5 are relevant now? Are there any specialized tasks where they might still excel over the newer models like Claude Sonnet or Opus or GPT-5.5	It depends on the cost, accuracy and response time. Suppose for some tasks you only care about fast response time and less cost but accuracy not matter much, those kind of cases older models are good enough. But if you have some tasks that require complex reasoning then newer models are better
is it same with encoder model?	Yes. The same principle applies to encoder models. An encoder is built by stacking identical encoder blocks, each containing one Multi-Head Self-Attention layer and one Feed-Forward Network. The number of encoder blocks is a design choice based on the model size and available compute.

From a production standpoint, you should try to avoid CoT prompts on Reasoning Models because it wastes your tokens and can confuse them sometimes. -> But wouldn't defining what chain of steps (thoughts) to follow ground the reasoning model and stop it from wandering of in random reasoning direction and save tokens in the long run?	I Knew this question would be coming, good that you asked, You would be 100% right if we were talking about a standard model like GPT-4o or Claude Sonnet! But reasoning models like o1 are completely different. Their internal thinking isn't a straight line, it involves exploring multiple paths, backtracking, and correcting its own errors. If you force a strict human "step-by-step" path onto it, you are basically micro-managing a senior developer. You constrain its highly optimized search process and usually force it to perform worse also wasting more tokens. To ground a reasoning model in production, you shouldn't tell it how to think. Instead, just give it incredibly strict goals, facts, and boundary constraints, and let it figure out the best path on its own. The trick is just in how we stop it from wandering Take Boundary Constraints for example, here you don't tell the model how to solve the problem, you just tell it what not to do. or you can give tasks like task 1 then task2 something like that
When models are trained on large corpus like wikipedia, book corpus, internet data these are unlabelled data. Then how the loss is calculated in training as we don't have actual values ?	Although the data is unlabeled, the text itself provides the supervision, which is why this is called self-supervised learning. For example, GPT is trained by predicting the next token. Given "The capital of France is", the next token "Paris" is already present in the training text, so it becomes the target. The model's prediction is compared with the actual next token, and the cross-entropy loss is calculated.
maybe my question is more about instructions than chain of thought, the question had a gap in logic	Good Good, you got the exact right distinction! There is a difference between providing clear instructions (which are absolutely required) and forcing a specific chain of thought (which backfires on reasoning models).
For decode only models, when we give context, i think there is role for encoder, why there is no encoder?	You can think of the self attention of the decoder is doing the encoder work. It understands the context while generating
In decoder only model, we don't have multi head attention and also cross attention...how this will build context now?	To clarify a quick misconception, decoder-only models actually do use Multi-Head Attention. The key difference is that they use Masked (or Causal) Multi-Head Attention, meaning a word is mathematically restricted to only look at the words that came before it. You are completely right that they do not have Cross-Attention, because there is no separate encoder to talk to. They build context entirely through self-attention. When you send a prompt, the model reads it left-to-right, and every single token absorbs the meaning of all the previous tokens. By the end of your prompt, the model has built a massive, mathematically dense memory of everything you just said, which it uses to predict the very first generated word.

In production system, will it be better to combine BERT for existing information extraction and then use GPT for generation? Or we will have to use decoder only model?	Yes, combining them is exactly the industry best practice. If you try to use a decoder-only model (like GPT) for everything, you are forced to paste all of your raw documents into the prompt every single time. This will massively spike your API token costs and make your application incredibly slow. Instead, the production standard is modularity where we use a fast, cheap encoder model (like BERT) to search or extract only the exact relevant information from your database, and then pass just that condensed information to the expensive decoder model to generate the final answer. You use the cheap model for the heavy lifting, and the expensive model just for the final step.
is claude decoder only model, why its popularity had significantly increased recently compared to others?	Yes. Claude is widely believed to be based on a decoder-only Transformer architecture, similar to GPT, although Anthropic has not publicly disclosed all of its architectural details.
Which model is best for text nowadays?	You see there is no single best model for all tasks. For general text generation, reasoning, and coding, GPT-5.5, Claude Opus, Claude Sonnet, and Gemini 2.5 Pro are among the top-performing models.
Why people went for decoder only model..??If we wanted to scale why we are not scaling transformers based model which has encode and decoder both. Simplicity can't be the only reason to go for decoder only model isn't it..??	Simplicity is only part of the reason. Decoder-only models scale better for open-ended text generation because their training objective (next-token prediction) is simple, self-supervised, and matches the generation task exactly. They can be trained on virtually unlimited unlabeled text, and their architecture is easier to scale to hundreds of billions of parameters. Encoder-decoder models are still widely used for sequence-to-sequence tasks such as translation, summarization, and speech recognition, where there is a clear input-output mapping.
For architecting and Coding what is the best model. If we have to learn new in demand concepts in IT. What is the best model to start with.	For open-source models, Qwen3 is currently one of the best all-round choices for coding, reasoning, and learning technical concepts. DeepSeek-R1 excels at reasoning and programming, while Llama 4 is a strong general-purpose model with a large ecosystem. If you have limited hardware, Gemma 3 offers an excellent balance of quality and efficiency. For most learners and developers today, Qwen3 is a great place to start.
what is the difference between context and context window . how it will affect the model response ?	The context is the actual content/information you provide to the model, but the context window limits the maximum amount of text the model can see at once
is prompt engineering focus around optimizing token usage to get the desired result?	Not exactly, that's part of it but not the main focus. Prompt engineering is primarily about crafting inputs (instructions, examples, context, formatting) to get the model to produce the most accurate, relevant, or desired output, things like using zero-shot vs few-shot examples, clear instructions, chain-of-thought cues, role assignments, or output format specifications. Token efficiency is a secondary, practical concern, since longer prompts cost more (API pricing is per-token) and consume context window space, but the core goal is steering the model's behavior and reasoning, not minimizing tokens. Good prompt engineering balances both: getting quality output while being reasonably efficient with the context you're given.
so the context window is our prompt text only ? or it contains the responses it generates also	At the beginning it is only the prompt you provided, but as it starts generating, those are also added to the context

So it means we cannot train transformers based model on unlabeled text..??. I think we can and I am not sure why it would underperform an decoder based model. Also I am not sure why it won't scale as good as decoder based model	You're right to push back, encoder-only models like BERT absolutely can and do train on unlabeled text, that's exactly what MLM and NSP are, self-supervised objectives on raw, unlabeled corpora (BookCorpus + Wikipedia). So unlabeled training isn't the limiting factor. The real difference is architecture-task fit, not training data type. BERT's bidirectional attention is great for understanding, but it can't generate sequentially since it sees the whole input at once with no causal masking. Decoder-only models scale better specifically for generation because their simpler, single-stack causal architecture aligns directly with next-token prediction at massive scale (GPT-3's 175B), making training more efficient and directly optimized for the generation objective itself, not because of labeled vs unlabeled data.
What is Temperature parameter used for in program	The temperature parameter controls the randomness of the model's output. A low temperature (e.g., 0.0–0.3) makes the model more deterministic and consistent, while a high temperature (e.g., 0.8–1.2) makes the output more diverse and creative. It will be taught in more detail in the coming slides.
what is temprature here, it was there in the sample code prof shared?	In a very crude way, temprature is used to set how much free text you want to generate. Like if temprature is low, you are telling the model to generate deterministic output, and if temprature is high then it can generate more creative output
Here in prompt word "think" used inside *. Will this have any impact on the prompt or output?	Some models are trained to follow such extra markers, so yes it can have impact
Here's a more natural version: In which situations is it recommended to use a high temperature?	Suppose you are writing a story then you want to be creative
What is the meaning of Proximity?	Proximity isn't a technical LLM or API term, it's something this particular prompt invented. Look at Rule 4 on the slide the model is told to end every reply with [Proximity: X/10], a 0 to 10 score for how close the user is to finding the bug themselves like hotter/colder in a guessing game. In the output you can see it gave [Proximity: 4/10], meaning the user's still a fair way off, and Rule 5 makes the score actually do something At 9+ the debugger gives a direct hint. The neat takeaway is that you can instruct a model to track and report its own running score entirely in plain English, with no special parameter though since the model is self-assessing, treat that number as a rough gauge, not a precise measurement.
In which situations is it recommended to use a high temperature?	Suppose you are writing a story then you want to be creative
Where does Prompt Eng lies in the architecture of Decoder model? Do they go in as new inputs and then re-train the model?	Yes there is a step called instruction fine tune, which is done after normal pre-training stage
can you give examples of some good Prompts?	You see what we call a "good" prompt is completely dependent on the domain you're using the LLM in. Ideally a good prompt is descriptive enough to give the model and idea of what task you are asking it to solve.
is this sentiment analysis correct example for this usecase.. prompting.. sentiment analysis is imp and cannot afford halicunation.. so i think systems involving sentiment can have a dedicated model and rest can use LLM .. why LLM for sentiment analysis .. it can be an on - device classifier.. but then how will that understand sarcasm and tone is my confusion.. that brings back to LLM .. so what is the optimal approach for sentiment analysis ?	It depends on the application. For straightforward sentiment analysis (e.g., positive/negative/neutral), a small, dedicated classifier such as BERT is often preferred because it is faster, cheaper, and more reliable. However, when the text contains sarcasm, irony, mixed emotions, or requires understanding broader context, an LLM generally performs better. In production, a common approach is to use a lightweight classifier for most requests and fall back to an LLM only for ambiguous or complex cases, balancing cost, latency, and accuracy.

Do these models learn from our prompt also? I mean if I am giving an information in a prompt will it only use it for output generation or its internal training also?	When you use a deployed model like chatgpt, it is only use the information provided in the prompt to generate text not to train itself. Training is only done before deployment, they can use your data for training for the next version of the model, not in the already deployed model
is the greedy decoding just a prediction and not generation?	Greedy decoding is a generation strategy, not just a single prediction. At each step, the model predicts the probability distribution over the next token and selects the token with the highest probability (argmax). It then appends that token to the sequence and repeats the process until a stopping condition is reached. So, it generates text by making a sequence of deterministic next-token predictions.
what will happen wehn two or more token have the same probability in greedy decoding. How will it take decesion?	It is rare but possible, it depends on the implementation, some uses random like pick any one of them, some use some auxilary steps to tie brake between the two, like the one occurs more in the training corpus
How does context length influence prompt design, and are there strategies to help maintain reasoning quality in very long-context applications?	Longer context windows allow more information to be included in a prompt, but simply adding more text does not guarantee better reasoning. Very long prompts can dilute important information or cause the model to lose focus. To maintain reasoning quality, you'll need to keep prompts well-structured, place the most important instructions and relevant context near the beginning and remove irrelevant information.
Can you pls repeat idea behind greedy decoding?	Greedy decoding picks the single highest-probability token at each step, without considering alternatives or future consequences. It's fast and deterministic but can produce repetitive or suboptimal text since it never backtracks or explores other options.
Can you explain further on this, where does "instruction fine tune" phase lie in the decoder model architecture?	Small but important reframe Instruction fine-tuning isn't a location in the architecture but it's a phase in the training lifecycle and the architecture (the decoder layers and LM head) stays exactly the same What changes is the weight values inside it. The pipeline runs pretraining, then instruction fine-tuning, then RLHF. So instruction tuning sits between pretraining and RLHF.
Why we picked arrived and the only and what means to K?	Arrived was not chosen arbitrarily. It was one of the top-K most probable first tokens predicted by the model. Here, K is the beam width, which is the number of candidate sequences retained at each decoding step. In this example, K = 2, so the model keeps the two best candidates ("arrived" and "the") and expands only those in the next step.
whay do we have only 3 decoding time steps ?	The three decoding time steps shown in diagrams are just for illustration. In practice, there is no fixed number of decoding steps. The decoder generates one token at a time and continues until it produces an end-of-sequence (EOS) token or reaches the maximum output length. So, a real model may perform tens, hundreds, or even thousands of decoding steps depending on the task.
How the EOS has been decided	Model naturally learns EOS probability through next token prediction
in beam search how does the the final output is derived?	In beam search, the model keeps track of the top-k most probable sequences (called the beam width) at each generation step, instead of just one. At every step, it expands each of the k sequences with possible next tokens, computes cumulative probabilities for all candidates, then keeps only the top-k overall sequences, pruning the rest. This continues until all sequences hit an end token or max length. The final output is the sequence among the surviving k with the highest overall cumulative (log) probability score, balancing exploration of multiple paths against the greedy approach's single-path limitation.

in beam search/Greedy Decoding, why do we have only 3 decoding time steps ?	3 decoding time steps were shown simply to keep the example simple and easy to visualize, not because beam search or greedy decoding is limited to 3 steps. Each step represents one token being generated, so showing 3 steps lets the diagram clearly illustrate how candidates expand and get pruned (in beam search) or how the single best token is picked (in greedy decoding) without cluttering the visual. In practice, decoding continues for as many steps as needed, until EOS is generated or a max length is reached, the 3-step diagram is just a toy illustration of the mechanism, not a fixed rule of the algorithm.
I current decoder only models with billion parameter are good enough to perform encoder only models task, like classification, sentiment analysis and etc. Then why encoders only models are in usage today, Is it due to customized dataset train, specific tasks, costing?	yes correct it is about cost, latency and adaptability tradeoff
How does causal (masked) attention influence a decoder-only model's reasoning capabilities compared to bidirectional attention?	Causal masking means each token can only attend to tokens before it (the future is masked out), while bidirectional attention lets every token see the whole input both directions. Bidirectional attention is a reader, it sees the whole page at once, great for understanding tasks like classification and embeddings where you have the full input up front. Causal attention is a writer, it only has access to what it's written so far and commits to the next word. Reasoning, it is much more like writing step by step than absorbing a page in one glance. A causal model generates one token at a time and conditions on its own previous output, it gets a scratchpad, it writes intermediate reasoning steps as tokens and attends back to them, which is exactly why chain-of-thought works and why a bidirectional single forward pass can't do the same thing. This causal + autoregression is what unlocks multi-step generative reasoning which is why every frontier generative LLM is decoder-only
In enterprise AI generates RCA reports. Should it use Greedy Decoding or Sampling? Why?	For RCA reports, greedy decoding (or a very low-temperature decoding strategy) is generally preferred because the output needs to be consistent, factual, and reproducible. Sampling introduces randomness, which is useful for creative writing but can lead to inconsistent or less reliable reports.
what is temperature here?	Temperature rescales the logits before the softmax is applied: $P_i = \exp(z_i / T) / \sum_j \exp(z_j / T)$ where z_i is the logit for token i, T is the temperature, and P_i is the probability of selecting token i. Lower values of T make the probability distribution sharper, favoring the highest-probability token, while higher values make the distribution flatter, increasing randomness. As T approaches 0, the output approaches greedy decoding.
What is the range for temperature? From previous slide I thought it should be between 0 and 1	Yes correct. 0->low, 1-> high temperature
In my Jira/ServiceNow incident summaries. Would I choose temperature = 0.2 or temperature = 1.5? Why?	You should choose temperature = 0.2 because incident summaries need to be consistent, factual, and reproducible. A low temperature minimizes randomness and reduces the chance of introducing unnecessary variations or hallucinations. A temperature of 1.5 is better suited for creative tasks such as storytelling or brainstorming, where diverse outputs are desirable.

More temperature means more time consuming?	Not essentially. Temperature has very little impact on inference time. It only changes how the next token is selected from the probability distribution after the model has computed it. The main factors affecting response time are the model size, input/output token length, hardware, and decoding strategy
how the outputs are different when the temp differs ?>	It is like low temperature: near deterministic, very focused output mid temperature: Balanced, factual natural responses, but has room for randomness or choices high: random, sometimes incoherent responses
Is temperature necessary or applicable only for Decoder only model? does BERT have a temperature? since it doesn't generate..	Temperature is primarily used for text generation, so it is mainly applicable to decoder-only models (e.g., GPT, Llama) and encoder-decoder models (e.g., T5) when they generate text. BERT does not have a temperature parameter during inference because it does not generate text token by token. Instead, it predicts masked tokens or outputs classification scores, where the highest-scoring prediction is typically selected directly.
Then how come temperature was set > 1 on previous slide?	I guess it depends on the model use. Different models use different ranges but the concept is the same. You need to check the model details before using them
In the previous slide it was said temperature belongs to {0,1} but the current slide temperature is reaching 1.8 why is that..???, What is the range of temperature	I guess it depends on the model use. Different models use different ranges but the concept is the same. You need to check the model details before using them
why the outputs are different when the temp differs ?> what's the logic behind and what technique it uses to generate the output	temperature modifies probabilities before sampling. Like when the temperature is low: it amplifies differences and when the temp is high it shrinks differences between tokens
what is the real life use case where we use top k sampling	Top-k sampling is useful in applications where the output should be natural and varied but still coherent. Tasks like story-telling, creative writing etc. can use them. By restricting the choice to the top K most likely tokens, it avoids selecting very unlikely words while still producing more diverse outputs than greedy decoding.
In what scenario we should use Top-K and in what scenarios we should use Top-p(Nucleus) Sampling?	Default to top-p for almost everything, it adapts to the model's confidence (narrows when the model is sure, widens when it's uncertain), which is what you want for chat, assistants, and general generation, and it's the more common production choice. Use top-k when you specifically need a hard, predictable cap on the candidate pool. Top-p's pool size varies with the distribution (could be 2 tokens or 200), so when you want bounded behavior top-k is the tool.
I understand that temperature only changes token selection, and not the model's knowledge. So why does increasing the temperature sometimes make the model give weaker or less accurate reasoning?	Temperature does not change the model's knowledge or reasoning ability. It only changes which tokens are sampled from the probability distribution. At higher temperatures, lower-probability tokens are selected more often, making each generation step more random. Since reasoning is built token by token, a few suboptimal token choices early in the process can lead the model down a weaker reasoning path, even though the underlying knowledge remains unchanged.
with Top K or Top P, do we still need temperature tuning?	Yes, temperature and top-k/top-p serve different purposes and are typically used together, not as substitutes. Temperature reshapes the entire probability distribution (sharpening or flattening it) before any token selection happens, while top-k/top-p restrict the candidate pool for sampling after that distribution is shaped. For example, you could have low temperature with top-k, very focused and limited to plausible options, or high temperature with top-p, creative but still avoiding the long tail of nonsensical tokens. Most production systems (like OpenAI's API) tune both together, temperature controls overall randomness, top-k/top-p controls which tokens are even eligible to be picked.

by setting the limit for max_tokens , are we at a risk of missing the best prediction?	Not exactly "missing the best prediction," but yes, there's a real risk: if max_tokens cuts off generation before the model reaches a natural EOS, you get truncated, incomplete output, a cut-off sentence or unfinished reasoning chain, even if the model was on track to produce a high-quality, complete response. This is especially risky for reasoning models or long-form generation, where the best answer needs more tokens to fully form. So max_tokens doesn't affect *which* tokens get predicted at each step, it just limits *how many* steps you allow, potentially truncating a good answer before completion.
In Top P we decide top tokens based on their probability. How do we decide top tokens in Top K?	In Top-K, instead of using a probability threshold, you simply pick a fixed number, k, of tokens, the k tokens with the highest individual probabilities, regardless of what their cumulative probability adds up to. For example, if k=5, the model looks at the full probability distribution over the vocabulary, ranks all tokens by probability, and keeps only the top 5, then renormalizes and samples from just those 5. The key difference: Top-K uses a fixed count, always exactly k candidates, while Top-P (nucleus sampling) uses a dynamic count, however many tokens are needed until their cumulative probability reaches p, so the candidate pool size can vary depending on how confident or uncertain the distribution is at that step.
can hallucinations be classified as false negatives	A hallucination occurs when an LLM generates information that is false, unsupported, or fabricated. A false negative is an evaluation error where the correct answer exists, but the model fails to identify or predict it. However, a hallucination could indirectly lead to a false positive or false negative in a downstream classification task, depending on the context. By itself, a hallucination is not classified as a false negative.
top P sampling always depends on threshold value given at prompt time?	Yes. Top-p (nucleus) sampling depends on the top_p threshold specified at inference time. At each decoding step, the model sorts tokens by probability and selects the smallest set whose cumulative probability is at least p (e.g., p = 0.9). The next token is then sampled only from this dynamically selected set. Different values of top_p change how many candidate tokens are considered.
if model is wrong ,even top P also doesn't work	Yes. Top-p sampling cannot fix an incorrect model prediction. It only changes how the next token is selected from the model's predicted probability distribution.
what happen in top K vs top P can u please explain with real life example? what used when? who use it? model?	Example: prompt "The weather today is..." Suppose probabilities are: sunny (40%), cloudy (25%), rainy (15%), terrible (10%), and a long tail of unlikely words. Top-K (k=3): only considers sunny, cloudy, rainy, ignoring everything else, even if "terrible" had decent probability, fixed count regardless of distribution shape. Top-P (p=0.8): keeps adding tokens until cumulative probability hits 80%, so it might include sunny, cloudy, rainy, and terrible if needed to reach that threshold, adapting dynamically to how confident the distribution is. Top-P is generally preferred in modern production systems (OpenAI's GPT models, Anthropic's Claude API) since it adapts better to context, very confident predictions get few candidates, uncertain ones get more. Top-K is simpler, used in some open-source/research setups (early GPT-2 sampling experiments) but less adaptive.
Why do LLMs still hallucinate even after instruction tuning and reinforcement learning?	Because instruction tuning and RLHF change how the model behaves (its tone, helpfulness, and format) Not the underlying fact that it predicts plausible-sounding tokens rather than retrieving verified truth. A vanilla model has no source of truth at inference, just lossy knowledge compressed into its weights, so when it doesn't precisely know something it interpolates a plausible answer instead of a real one.

If an AI agent needs to retrieve live information from enterprise systems like ServiceNow, k8s, or certificate management platforms, is function calling the recommended architecture, or are there cases where MCP or another integration approach is preferred?	Function calling is the core mechanism, the model invokes a specific function with structured arguments. MCP is a standardized protocol built on top, providing a common interface so each tool (ServiceNow, k8s, cert systems) doesn't need custom integration code. For a single integration, direct function calling is simpler. For connecting many enterprise systems with reusable, standardized connectors, MCP is preferred, it acts as a universal adapter layer across tools and models.
I am jumping ahead maybe, however to understand this we can use openAI api to call any local llms as well? also where does MCP fit in this? can we also use MCPs for basic chat completion as shown in this exmaple?	On MCP, Its a different layer, and no, it's not for basic chat completion. Chat completion is how you talk to the model MCP (Model Context Protocol, an open standard from Anthropic) is how a model or app reaches out to external tools and data in a uniform way.
let's say im writing a code for a basic chat completion. if i want the user prompt to be variable (i.e, user inputs a different prompt every time), in the messages dict in "role": "user", can i use an input() function in the "content" key?	Yes. You can use input() to make the user prompt dynamic. The value returned by input() can be passed as the content of the "user" message, allowing a different prompt to be sent to the model each time the program runs.
How does internal working of the client.chat.completions.create() works? How does it map the mentioend model and take that for response creation?	Let me break this down step by step on what happens in the background client.chat.completions.create() is a thin network wrapper, not where the model runs. It serializes your arguments into a JSON body (model, messages, temperature, etc.) and sends an HTTPS POST to the API server with your key in the header. a dispatch layer looks it up in a model registry and maps it to the right deployed weights (which cluster, which version), essentially a dictionary lookup. The chosen server then formats your messages into the model's chat template, tokenizes, runs the autoregressive generation with your decoding params, and sends back JSON that the SDK turns into the Python object you read.
what does the temperature 0.7 indicates in the code shown?	It is telling the model to be balanced sweet spot between deterministic and creative responses
how does the model resolve if the content generation conflicts with any roles? or does it decide which role to play when it generates?	The model does not choose a role on its own. During inference, it receives all the messages (system, developer, user, assistant) together as context. If there are conflicting instructions, it follows a priority order: system instructions > developer instructions > user instructions > previous assistant messages. The model generates a response that best satisfies the highest-priority instructions while ignoring or minimizing lower-priority conflicting ones.

didn't understand the purpose of role in the message list...how is it predicting the next word based on role for each sentence?	Roles (system, user, assistant) in the message list aren't separate mechanisms for the model, they're just formatted as part of the text sequence using special tokens/templates before being fed into the same next-token prediction process. For example, the messages get converted into a single formatted string like: <[system]> You are a helpful assistant <[user]> What is BERT? <[assistant]>, then this entire sequence is tokenized and processed normally. The model learns during training (via instruction tuning/RLHF) to recognize these role markers as context cues, text following <[system]> sets behavior/tone, text after <[user]> is the question, and text after <[assistant]> is where it should generate a helpful response matching that learned pattern. So roles don't change the underlying mechanism, the model still predicts next tokens the same way, but it's learned to behave differently based on which role-tagged section it's generating into.
Can you please explain how temperature works mathematically?	Temperature scales the logits (raw model outputs) before applying softmax: $P(\text{token}) = \text{softmax}(\text{logits} / T)$. Dividing by T changes how "peaked" or "flat" the resulting probability distribution is. When T is low (e.g., 0.3), dividing by a small number makes the logits larger in magnitude, this exaggerates differences between them, so after softmax, the highest-logit token gets pushed very close to probability 1, and others shrink toward 0, nearly deterministic. When T is high (e.g., 1.8), dividing by a large number shrinks the logits closer together, after softmax, probabilities become more uniform across tokens, so even lower-probability tokens get a real chance of being sampled, increasing randomness. At T=1, logits are unchanged, giving the original, unmodified distribution the model naturally learned.
How to decide if we have to use Top K or P or Temperature?	It depends on the level of control you need. Use temperature when you want overall control of randomness/creativity across the whole distribution, simplest, most common starting point. Use Top-K when you want a hard cap on candidate diversity regardless of confidence, useful for simpler, more predictable tasks. Use Top-P (nucleus sampling) when you want adaptive diversity, more candidates when the model is uncertain, fewer when it's confident, generally preferred in production (GPT, Claude) since it's more context-sensitive. In practice: start with temperature alone for most tasks; add top-p if you need finer control over creativity while avoiding nonsensical tokens; use top-k for simpler, more constrained generation needs.
How is LLM going to introduce new token - prompt, context into it...how it identify it?	LLMs don't actually encounter truly "new" words at the character level, they use subword tokenization (like BPE - Byte Pair Encoding, or WordPiece for BERT). Any unfamiliar word gets broken down into smaller known subword units or even individual characters that already exist in the fixed vocabulary. For example, an unfamiliar word like "ChatGPT" might get split into tokens like "Chat" + "G" + "PT", all of which already exist in the vocabulary, even if the full word never appeared during training. This way, the model can represent virtually any input text, even brand-new words, names, or technical jargon, using combinations of existing subword tokens, without needing a literal "new" vocabulary entry. So new prompt context isn't "identified" as new, it's decomposed into familiar subword pieces, then processed normally through embeddings and attention layers like any other token sequence.

can you explain what is Temperature mean here?	Temperature controls the randomness of the model's output by scaling the logits before softmax: $P(\text{token}) = \text{softmax}(\text{logits} / T)$. Low temperature (e.g., 0.3) sharpens the distribution, making the model almost always pick the most probable token, deterministic, focused, predictable output like "Paris is the capital of France." Default temperature ($T=1.0$) keeps the original learned distribution unchanged, balanced creativity and coherence. High temperature (e.g., 1.8) flattens the distribution, giving lower-probability tokens a real chance of being picked, more random, creative, but can become incoherent, e.g., "Paris is cheese-dream cobblestone forever!" So temperature doesn't change which tokens are possible, it changes how confidently the model commits to the most likely ones versus exploring less likely alternatives.
Hi It is just explained that decoder-only models work entirely by next token prediction. If the model is just guessing the next likely words, how can it reliably 'call a tool' ? Does it actually run Python code or call API, or is it still just playing a game of probability?	it's still fundamentally next-token prediction, there's no separate "tool-calling brain." The model just predicts that, given the context, the most probable next tokens form a structured output (like JSON) matching a function schema, e.g., <code>{"function": "get_weather", "args": {"city": "Mumbai"}}</code>, purely because it learned during fine-tuning that this pattern is the appropriate response to certain prompts. The actual execution (running Python, calling an API) happens outside the model entirely. The application/orchestration layer parses this predicted text, recognizes it as a structured function call, executes the real function/API call, then feeds the result back into the model's context as new input text. The model then continues next-token prediction using that result to generate its final response.
Is Tool Calling same or similar to MCPs?	Tool calling is a mechanism and the MCP is the standard built on top of it. It is a protocol about how tool call will be performed
what are tools we are passing here in the LCM calculation example?	one tool is defined: "compute_lcm", with type "function". It includes a description ("Computes the Least Common Multiple of two integers"), and a parameters schema specifying it needs two required integer inputs, "a" and "b", each with their own description. This schema tells the model exactly what arguments to extract from the user's question and how to format them. Separately, the actual Python function <code>compute_lcm(a, b)</code> is implemented using <code>math.gcd</code>, this is the real code that executes when called. So the "tools" list is just the model-facing contract (name, description, expected parameters), while the function definition below it is the actual implementation that runs once the model predicts a matching function call.
what is difference in MCP and tool?	Tool calling is a mechanism and the MCP is the standard built on top of it. It is a protocol about how tool call will be performed
How the total tokens are calculated ?	It depends on the model tokenizer. Tokenizer splits your input text to multiple tokens and also while generating it generates the tokens. So total tokens is the sum of input and output token generated. You can check the tokens for openai models here: https://platform.openai.com/tokenizer
where RAG comes into picture? In prompting, can you pls explain	Suppose you are using LLMs to your organisation and suppose your company work in legal domain. Now the existing models may not have legal information much then what you can do is for your query find the relevant documents from your company database and then put some of the top matched documents as the context for further LLM processing

what are tools we are passing here in the LCM calculation example?	Yes in the slide 64, compute_LCM is defined and pass as tool
are we not passing compute_LCM as function?? in the tools?	
at an enterprise level, there are huge numbers of tools and its impossible to hard code them in the system. what are the ways to handle this in real world scenarios? best practices and design considerations ?	Youre Right, at scale you never put all tools in the prompt, because cost, latency, and especially selection accuracy all degrade as the tool count grows. The headline solution is tool retrieval, essentially RAG for tools. We embed every tool's description in a vector index, and at runtime retrieve only the handful most relevant to the current query, so the model only ever sees 5 to 20 tools no matter how many thousands exist. Another way could be, Hierarchical selection (pick a category first, then tools within it), One More could also be multi-agent routing (a supervisor delegates to specialized sub-agents each owning a small toolset)
Most models are geared for understanding textual content. However a great number of research based use cases need analysis and understanding of numerical data. Understanding numbers as text is clunky. Are there any LLM's specially geared towards numbers and math and science research ?	Your premise is half-right, Numbers really are clunky for LLMs, because tokenization means the model sees number tokens, not magnitudes (it has no built-in sense that $1001 > 1000$), and it has no arithmetic unit, so it pattern-matches what an answer looks like rather than computing it. Current strong ones include open models like DeepSeek-R1 and Qwen's thinking models, and frontier closed models (GPT-5.x, Claude Opus, Gemini) lead the hard math benchmarks; there are also math-tuned fine-tunes like Mathstral, WizardMath, and proof models like DeepSeek-Prover, though their tooling is less mature. For actual numbers, don't trust the model's mental math, pair it with a tool, having it write and run Python or call Wolfram Alpha for exact computation while the LLM handles the reasoning.
where RAG comes into picture? In prompting, can you pls explain	Suppose you are using LLMs to your organisation and suppose your company work in legal domain. Now the existing models may not have legal information much then what you can do is for your query find the relevent documents from your company database and then put some of the top matched documents as the context for further LLM processing
If my prompt already says "Answer in only two lines," do I still need to set max_tokens? How do these two parameters complement each other?	max_tokens is the hard limit, model will check each time while generating while it passes the max limit or not. While giving "Answer in only two lines," in the prompt is a soft signal to the model, while processing this model need to understand your intention, it is like context
Is the tool calling done internally by the model defined or user has to explicitly make a call? If its generated internally then everytime it will keep genearting the new method as per the prompt?	You need to define the tool available for the model to use. Then while processing your input model can only use out of those tools.
In Decoder only model We have only masked multi head attention wRT to predicted text. But how the input attention is treated?	There's no separate "input attention" in a decoder-only model the prompt and the generated text are one single concatenated sequence, and the same masked (causal) self-attention applies to all of it uniformly. The input is processed causally too, left-to-right, a token inside your prompt only attends to earlier prompt tokens, not later ones, so the model does NOT read your prompt bidirectionally the way BERT would. It reads it under the same left-to-right cover as the text it writes. And there's no cross-attention at all, that only exists in encoder-decoder models. Decoder-only has exactly one attention type.

this question is confusing? are all transformer based models are encoder and decoder based? i tot transformer is an architecture which has attention isnt BERT also a transformer based model?	Your instinct is right BERT IS a Transformer-based model, and no, not all Transformer models are encoder-decoder. The confusion is one word doing double duty. "Transformer" means the building-block style and in that sense BERT, GPT, LLaMA, and T5 are all Transformer-based.
so in case of RAG, we fetch chunks by querying vector database and those chunks are embeddings? and do we convert embedding back to text before going to LLM for processing?	So basically these are vector databases, you can find the actual snippet using the embedding and pass the actual snippet as prompt
what is actually logits? is it what we apply in softmax?	Yes, exactly Logits are precisely what you feed into softmax. They're the model's raw, unnormalized scores, one per vocabulary token, that come out of the final layer before softmax converts them into probabilities.
dynamic on probabiltiy means? in top P?	Can you please explain the question a little so that we can provide response better
How does causal (masked) attention influence a decoder-only model's reasoning capabilities compared to bidirectional attention? Here reasoning ability mean answering questions	In the decoder only model you can not use bi directional attention because in each next token generation you only have the access of previous generated tokens. There is no such future tokens
do we have token limt to api key?	20 calls per minute i guess, if you hit any limit, you'll see the messege in error
Whats Hugging Face Router ?	The Hugging Face Router is HF's OpenAI-compatible endpoint (router.huggingface.co/v1) that fronts their "Inference Providers" It puts 15+ inference backends behind one API and one token. It's the cross-provide.
how to get the hugging face token key?	You can go to https://huggingface.co/settings/tokens
Why Base_url is required while creating client?	base_url specifies which API endpoint your requests get sent to
can you explain what are roles?	Roles label who is "speaking" in each message of the conversation, so the model knows how to interpret and respond to that text. The three main roles are: system (sets the model's behavior/instructions, e.g., "You are a helpful assistant"), user (the actual question or input from the person), and assistant (the model's own previous responses, used to maintain conversation history). In the LCM example, you saw {"role": "user", "content": "What is the LCM of 84 and 120?"}. The model is trained (via instruction tuning) to recognize these role tags as context cues, text after "system" sets tone/rules, text after "user" is the query to respond to, and text after "assistant" represents the model's own prior turns in multi-turn conversations.

how to fix this error:----- <pre> AuthenticationError Traceback (most recent call last) /tmp/ipykernel_773/751006345.py in <cell line: 0>() 1 # Send a prompt to the model ----> 2 response = client.chat.completions.create(3 model=MODEL, 4 messages=[5 { 3 frames /usr/local/lib/python3.12/dist-packages/openai/_base_client.py in request(self, cast_to, options, stream, stream_cls) 1103 1104 log.debug("Re-raising status error") -> 1105 raise self._make_status_error_from_response(err.response) from None 1106 1107 break AuthenticationError: Error code: 401 - {'detail': 'Unauthorized'} </pre>	check if api key is correct or not!
model being used is :Using model: Qwen/Qwen2.5-7B-Instruct-AWQ	yes
what if I dont provide roles?	If you don't include the "role" field, you'll get an error, it's a required field in the message object, the API expects each message to be a dictionary with both "role" and "content" keys. Skipping it isn't optional formatting, it's a structural requirement of the API schema.
thanks, r we supposed to know the base_url?	use the same rul as the notebook, with you own api key provided by upgrad. is that not working?
Does the max_tokens limit mean the model will summarize the response and stop within that limit, or will it stop immediately once it reaches, for example, 200 tokens?	max token is a hard limit and will stop immedietely when the limit reaches
How to validate for key its peinted:Using model: Qwen/Qwen2.5-7B-Instruct-AWQ	<pre> check with this: import os key = os.environ.get("OPENAI_API_KEY") # or whatever env var your provider uses if key: print(f"Key found, starts with: {key[:6]}... length: {len(key)}") else: print("No key found in environment.") </pre>
how to determine the temperature here?	It depends on the task, if you asking factual question like what is ingredients of a medicine go for lower temparature like 0-0.5 But suppose you writing some report or story then go for higher temperature

can you give a brief about role , i.e how does system/user/assistant differs when model responding?	System role sets the model's overall behavior, tone, and instructions before the conversation starts, e.g., "You are a helpful coding assistant who responds concisely." It's not something the user typed, it shapes how the model interprets everything after it. User role represents actual input/questions from the person interacting with the model, this is what the model is meant to respond to at each turn. Assistant role represents the model's own previous responses, included in multi-turn conversations so the model has memory of what it already said, maintaining context and consistency across the dialogue.
What is the use of this prompt. You should not ask the LLM model to use the LLM model	which part exactly are you referring to?
what is temperature and how it impact the API calls of LLM	Temperature controls the randomness/creativity of the model's output by scaling the logits before softmax: $P(\text{token}) = \text{softmax}(\text{logits} / T)$. Low temperature (e.g., 0.2-0.3) makes output deterministic and focused, the model almost always picks the highest-probability token, useful for factual Q&A, code generation, or tasks needing consistency. Default temperature (1.0) keeps the natural learned distribution, balanced. High temperature (e.g., 1.5-1.8) flattens the distribution, increasing randomness, useful for creative writing, but risks incoherent output. In API calls (like OpenAI's), temperature is a parameter you pass directly, e.g., temperature=0.7, it doesn't change which tokens are possible, just how confidently the model commits to likely choices versus exploring alternatives. It also affects reproducibility, same prompt at low temperature gives near-identical outputs across calls, while higher temperature gives varied outputs each time, important for testing/debugging vs. creative applications.
please provide some link so that we can build our understanding for temperature in API calls	https://medium.com/@kelseyywang/a-comprehensive-guide-to-llm-temperature-%EF%B8%8F-363a40bbc91f
This part PROMPT = "" Suggest one novel research direction using Large Language Models. Keep the answer under 50 words. ""	it is asking for research directly which could be tackled by LLMs. Type of research where using an LLM can benefit us.
Are there any other API other than chat-like APIs provided by OpenAI or other labs?	no, same. these are OpenAI keys.
Let's say we give labelled numerical feature vectors as a RAG prompt to an LLM. Would that architecture be more robust and efficient (computationally and response time) compared to giving it ML model and training on the data. A general purpose LLM should be able to do just as well as a trained model ????	No, this generally wouldn't be more robust or efficient than a trained ML model for structured/numerical tasks, and a general-purpose LLM with RAG won't match a properly trained model's performance on that data.
Tool calling vs RAG? plz explain	Suppose you need current weather info or stock price of certain stock you need to use tool calling because up-to-date information is not present in the pre-trained model. On the other hand if you want to use your company-specific data to infer something what you can do, query your company database, extract related documents and pass as prompt context to generate more accurate response. So here RAG is needed.

What has now been explained is called context rot. right? Usually summarizing the context window is one good way for this. Should we use an encoder based model to summarize the context and then send the summarized context window to the LLM? Would this be cost effective? or there are better ways to handle this...?	Not quite, encoder-only models like BERT can't generate summaries, they lack a decoder, so they can't produce new text output. Summarization needs a generative model (decoder-only or encoder-decoder like BART/T5). In practice, context summarization is usually done by a smaller/cheaper LLM call itself, summarize older turns periodically, then feed that condensed summary plus recent raw messages to the main model. This is cost-effective since the summarizer model can be small and cheap, while the main model only handles shorter context. Other approaches include sliding window memory, retrieval-augmented memory (RAG over past conversation), or hierarchical summarization for very long contexts.
How does model decide which API to call while tool calling?	The model doesn't truly "decide" via separate logic, it's still next-token prediction. When you provide a list of tools (function name, description, parameter schema) in the API call, the model's context includes this information alongside the user's query. Based on patterns learned during fine-tuning, the model predicts that, given the query and available tool descriptions, the most probable output is a structured function-call response matching one specific tool's name and required arguments. The model doesn't truly decide, it's still next-token prediction. Given the user's query plus the tool descriptions/parameters provided in context, it learned during fine-tuning to recognize which tool's description best matches the intent, then generates a structured function-call output (name + arguments) for that tool instead of a normal text reply. Execution still happens outside the model, your code parses this output and runs the actual function.
what can be the other types in tool definition except function	Beyond the function type you define yourself, the main other category is provider-hosted built-in tools Common built-in types (names and availability vary by provider and change fast) web search, code execution / code interpreter, file search / retrieval, computer use (the model drives a virtual screen and keyboard), image generation, and remote MCP (connecting to external MCP servers). So the key distinction is who executes, with function tools the model requests and your code runs it and feeds the result back (full control, you build it), while with built-in tools the provider runs it server-side and the result comes back automatically.
Is there any specific way to write these tools? I mean current explanation is for determinant. What if we need something else. How to proceed for that?	Yes, there's a consistent pattern, you just swap out the specifics. Every tool definition follows the same structure: "type": "function", then a "function" object with name (unique identifier), description (clear, specific text the model uses to match user intent to this tool, the most important part for correct selection), and parameters, a JSON schema describing required/optional arguments with their types and descriptions. To create a new tool for anything else, follow these steps: write the actual Python function first (like <code>compute_determinant</code> using <code>numpy</code>), decide what inputs it needs and their types (string, number, array, object, etc.), then mirror those inputs in the parameters schema with matching names, and write a clear description so the model knows when to invoke it, e.g., for a weather tool, description might be "Get current weather for a given city," with parameters like <code>city</code> (string).

even if we set the temperature to be near to 0, and the tool is giving erroneous output, will the LLM still try to correct the output?	No, the LLM has no way to detect or correct an erroneous tool output, it has no independent way to verify correctness, it just trusts whatever text comes back in the "tool" role message and continues next-token prediction based on that input. Temperature affects only how the model samples its own generated tokens, it has nothing to do with validating external tool results. If your Python function (like <code>compute_determinant</code>) returns a wrong value due to a bug, incorrect logic, or numerical instability, the model will treat that wrong result as ground truth and generate its final response based on it, confidently presenting incorrect information without any awareness it's wrong, unless the error is glaringly inconsistent with the question itself (e.g., returning text instead of a number), in which case it might flag confusion, but it can't actually verify numerical correctness.
---	--